

git+github协作开发流程

0.协作开发核心原则

- **分支分离**：main分支仅用于发布稳定版本代码，禁止直接push到main分支；dev分支用于日常开发；feature-xxx分支用于单个功能开发。
- **频繁同步**：每次开发前，首先通过 `git pull origin dev` 拉取远程最新代码，开发完成后及时通过 `git push` 将代码推送到远程对应分支。
- **小步提交**：每次提交仅包含一个小功能或Bug修复，提交说明清晰。

1.团队成员克隆远程仓库

甲（仓库所有者）和乙都需要在 `Git Bash` 中输入如下内容，将远程仓库克隆到本地仓库：

```
1 | git clone 远程仓库地址
```

2.创建协作分支

为了保证main分支的稳定性，实际协作中通常不在main分支直接开发，而是基于dev分支进行协作。

甲在 `Git Bash` 中输入如下内容以创建dev分支：

```
1 | git checkout -b dev
2 | git push -u origin dev #将dev分支推送到远程dev分支，并设置上游为dev
```

乙拉取远程dev分支到本地仓库，并关联远程dev分支：

```
1 | git pull origin dev #拉取远程dev分支
2 | git checkout -b dev origin/dev #创建本地dev分支并关联远程dev分支
```

3.多分支并行开发核心流程

- 从dev分支创建feature-xxx分支
- 在feature-xxx分支上开发并推送代码
- 功能开发完成后，合并到dev分支

3.1 甲的操作（开发a模块）

```
1 git checkout dev
2 git pull origin dev
3 git checkout -b feature-a
4
5 ...(开发a模块)
6
7 git add .
8 git commit -m "新增a功能"
9 git push origin feature-la
```

3.2 乙的操作（开发b模块）

```
1 git checkout dev
2 git pull origin dev
3 git checkout -b feature-b
4
5 ...(开发b模块)
6
7 git add .
8 git commit -m "新增b功能"
9 git push origin feature-b
```

3.3 甲（仓库管理者）将以上两个功能分支合并到dev分支

```
1 git checkout feature-a
2 git pull origin dev #避免合并时冲突
3 git push origin feature-a
4 git checkout dev
5 git merge feature-a
6
7 git checkout feature-b
8 git pull origin dev #避免合并时冲突
9 git push origin feature-b
10 git checkout dev
11 git merge feature-b
12
13 git push origin dev
```

当然乙也可以进行如上操作。